

REQ 2 Design RationaleA. Teleportation Device Abstraction

Design 1: Implement teleportation logic directly inside each teleporting class (TeleportationTube, MagicCircle, AlienCube) by duplicating the move actor call and arrival effects inline within their respective allowableActions() methods.

PROS	CONS
Simple to implement, no additional interface needed. Each class handles its own movement logic in one place.	Violates DRY the actor movement call and arrival effect logic would be duplicated across all three teleporting classes.
All teleport logic is visible directly inside the class that triggers it.	Violates OCP adding a new teleportation device requires independently reimplementing movement logic with no shared contract to enforce correctness.

Design 2: Add a Teleporter interface with a single teleport(Actor, GameMap, Location) method. All teleportation devices (TeleportationTube, MagicCircle, AlienCube) implement this interface and define their own arrival effects inside teleport(). A shared TeleportAction delegates to teleporter.teleport() in its execute() method.

PROS	CONS
Satisfies SRP TeleportAction is solely responsible for invoking teleportation; each device class is solely responsible for defining its own arrival effects.	Teleporter is currently placed in game.grounds, which is a mild coupling smell since AlienCube (an item) also implements it.
Satisfies OCP the new teleportation devices only require a new class implementing Teleporter with no changes to TeleportAction.	
Satisfies DRY single TeleportAction class is reused across all three devices with zero duplication of menu.	

Design 1 has connascence of algorithm, because movement and arrival effect logic would be duplicated across all three device classes with no shared contract.

On the other hand, Design 2 reduces this to connascence of name on teleport and connascence of type on Teleporter, both compiler enforced. A mild connascence of position remains in the teleport(Actor, GameMap, Location) signature as all implementors must agree on parameter order. Lastly, a connascence of identity persists as each device class declares its own static Random, making malfunction behaviour difficult to test deterministically.

B. Door Tier Hierarchy

Design 1: Implement AluminiumDoor, IronDoor, and TitaniumDoor as completely independent classes each duplicating lock state management, clearance checking, alarm handling, and allowableActions() logic internally.

PROS	CONS
Simple, no abstract class needed, each door is entirely self contained.	Violates DRY, lock state (isUnlocked), alarm state (isAlarmed), clearance checking, and allowableActions() logic would be duplicated across all three door classes.
	Violates OCP fixing a bug in unlock logic (e.g. alarm relocking) requires changing every door class individually.
	Violates LSP without a shared supertype, doors cannot be substituted for one another where a generic Door reference is expected.

Design 2: Implement an abstract Door class extending Ground and implementing Unlockable and Alarmable. It owns all shared state (isUnlocked, isAlarmed, securityLevel) and behaviour (clearance checking, allowableActions(), alarm toggling). Subclasses only override the abstract unlockSideEffects(Actor, GameMap) method to define tier specific hazards or benefits.

PROS	CONS
Satisfies DRY lock state, clearance logic, and alarm handling are written once in Door and inherited by all three subclasses.	Extending Ground introduces tight coupling to the engine's ground hierarchy changes to Ground may require updating Door and all subclasses.
Satisfies OCP adding a new door tier requires only a new subclass overriding unlockSideEffects() with no changes to existing classes.	Subclasses must remember that unlockedBy() in Door already sets isUnlocked = true before calling unlockSideEffects(), requiring awareness of the parent's call order.
Satisfies LSP all three door types are interchangeable as Door references, and the Alarmable/Unlockable contracts are upheld uniformly.	

In Design 1 introduces connascent of algorithm. Lock state, clearance checking, and alarm logic would be independently reimplemented in each door class.

As compared to Design 2 which reduces this to connascent of name on unlockSideEffects and connascent of meaning on the security level integers (1, 2, 3), which all door classes and canActorUnlock() agree on implicitly with no single authoritative source. A deliberate connascent of execution exists in the Template Method: Door.unlockedBy() sets

isUnlocked = true before calling unlockSideEffects(), so subclasses must be aware the door state is already mutated when they receive control.

C. Toxic Waste as Permanent Ground

Design 1: Implement toxic waste damage by adding a poison like status to any actor that steps onto a corrupted tile, tracked inside the actor or via the existing PoisonStatus class.

PROS	CONS
Reuses the existing status structure from REQ1's poison system.	Misrepresents the mechanic toxic waste is a persistent environmental hazard tied to a location, not a temporary timed effect on an actor. A status expires; toxic waste never does.
	Requires the tile to reapply the status every turn, adding fragile reapplication logic.
	Does not accurately model the spec's intent that the ground itself is permanently corrupted.

Design 2: Implement ToxicWaste as a new Ground subclass that overrides tick(Location) to deal 1 damage per turn to any actor standing on it. AlienCube.teleport() calls location.setGround(new ToxicWaste()) on all adjacent tiles at the source location.

PROS	CONS
Satisfies SRP ToxicWaste is solely responsible for its own perturn damage logic; AlienCube is solely responsible for triggering the corruption.	ToxicWaste permanently replaces whatever ground was there before, with no way to restore the original tile. This is intentional as the spec but is irreversible by design.
Accurately models the spec corrupted ground is a permanent map change, not an actor status, and Ground.tick() is the correct hook for location based perturn effects.	
Satisfies OCP future ground types with perturn effects follow the same tick() override pattern with no changes to existing classes.	

Design 1 introduces connascence of timing. The tile would need to reapply a poison status every turn, coupling the ground and actor status systems to execute in the correct order each turn.

In Design 2 produces only a single connascence of execution within AlienCube.teleport(): the source location must be captured before moveActor is called, otherwise the wrong tiles are corrupted. This is localised to three consecutive lines in one method. Beyond this, ToxicWaste.tick() has no dependency on any other class's state or timing.

D. Magic Circle Peer Registry

Design 1: Give each MagicCircle a direct List<MagicCircle> reference to all other circles on the map, populating it manually at construction time in Overflow20.

PROS	CONS
Its direct, each circle can immediately query its peers without an intermediary object.	Violates DRY every circle holds a full copy of the peer list, meaning adding or removing a circle requires updating every other circle's list individually.
	Tight coupling between MagicCircle instances circles are aware of each other's concrete type rather than just their locations.

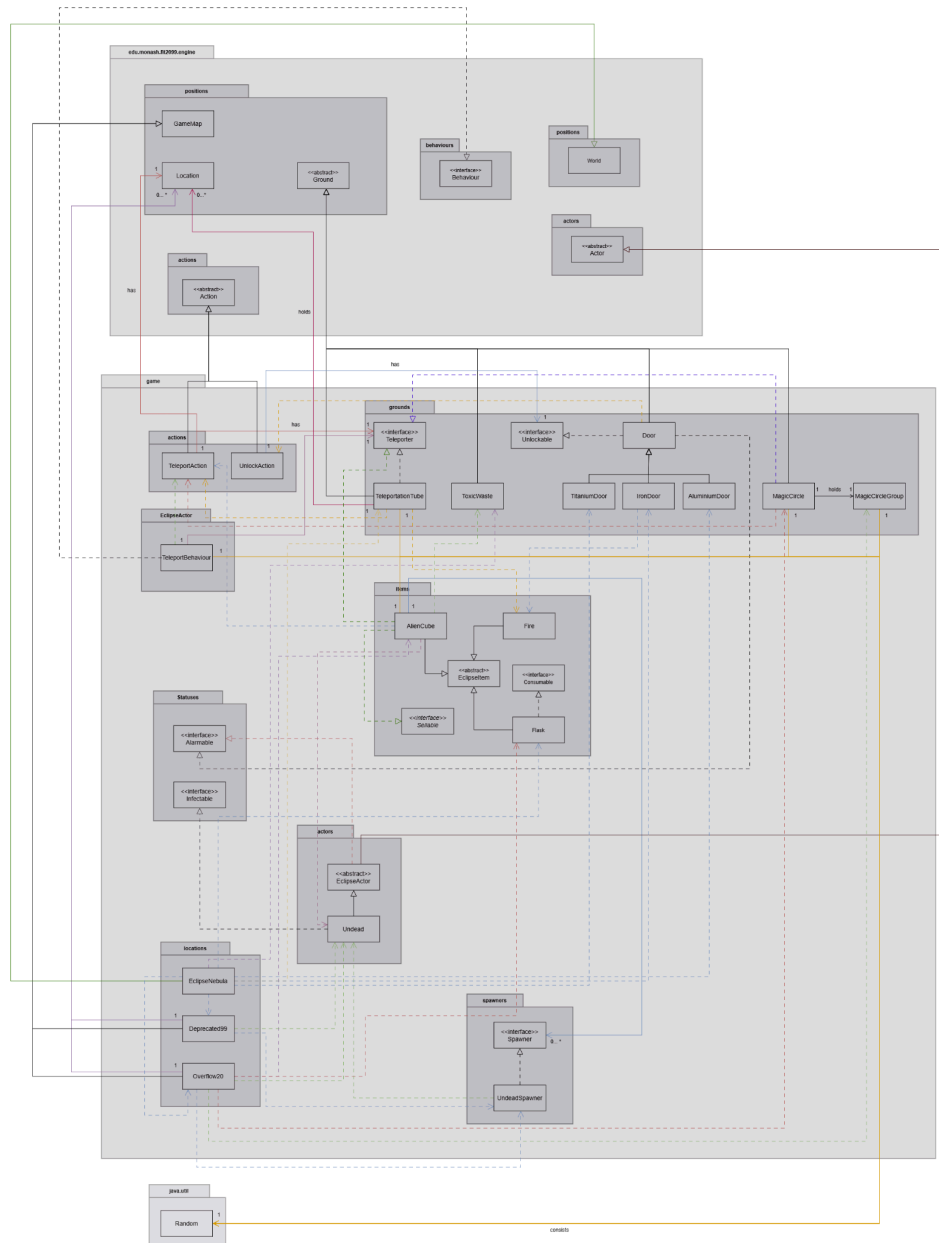
Design 2: Introduce a MagicCircleGroup registry class. Each MagicCircle holds a reference to the shared group and calls group.randomPeerExcluding(location from) to get a destination. Overflow20 registers each circle's location into the group after placing it on the map.

PROS	CONS
Satisfies DRY peer selection logic is written once in MagicCircleGroup.randomPeerExcluding() and reused by every circle in the network.	Overflow20 must remember to call circles.register() after placing each circle forgetting this silently results in a circle with no valid peers and no teleport action offered, which can be hard to diagnose.
Satisfies SRP MagicCircle is responsible for offering the teleport action; MagicCircleGroup is solely responsible for managing the peer list and selecting a random destination.	
Satisfies OCP adding more circles to the network only requires placing them and registering their location; no changes to MagicCircle or MagicCircleGroup are needed.	

Design 1 has connasence of identity each circle holds direct references to every other circle instance, multiplying the change surface with every addition.

Design 2 reduces this to one contained connasence of identity, all circles must share the exact same MagicCircleGroup instance, localised to Overflow20.setMagicCircles(). A connasence of timing is introduced as the noted con: setGround must precede circles.register() for each circle, as randomPeerExcluding() filters by map reference equality and a location not yet placed on a map will fail silently.

Summary



The diagram above shows the final design of REQ2, which applies Design 2 across all four decisions. The core principle driving every design choice in REQ2 is that each class owns exactly one responsibility, shared behaviour is written once and inherited or delegated rather than duplicated, and the system can be extended with new teleportation devices or door tiers without modifying any existing class.

The Teleporter interface establishes a shared contract across all three teleportation devices. Since each device has meaningfully different arrival effects, fire rings for TeleportationTube, flask spawning for MagicCircle, and toxic corruption for AlienCube; polymorphism through `teleport()` is more appropriate than a shared base class. TeleportAction depends only on the Teleporter abstraction, satisfying the Dependency Inversion Principle, and any future device

implementing Teleporter is automatically compatible with the existing action infrastructure (Open/Closed Principle).

The abstract Door class eliminates the duplication that would arise from three independent door implementations. Lock state, alarm toggling, clearance verification, and allowableActions() are defined once and inherited. The Template Method pattern is applied through the abstract unlockSideEffects() hook, Door.unlockedBy() defines the invariant sequence (set unlocked, then apply side effects) while each subclass fills in only what is unique to its tier. This means AluminiumDoor, IronDoor, and TitaniumDoor each contain only lines that distinguish them from one another.

ToxicWaste is modelled as a permanent Ground subclass rather than a reused actor status because the corruption is a lasting map mutation, not a timed effect on an actor. Using Ground.tick() as the damage hook correctly couples the effect to the location rather than to whoever happens to be standing there, and accurately reflects the spec's intent that the ground itself is irrecoverably changed.

MagicCircleGroup decouples circle to circle awareness behind a registry. Each MagicCircle depends only on the group abstraction rather than holding direct references to peer instances, satisfying the Dependency Inversion Principle. Peer selection and same map filtering logic live entirely within MagicCircleGroup.randomPeerExcluding(), meaning the rule that teleportation is within map only is enforced in exactly one place (DRY).